Software Requirements Specification (SRS)

25-26J-011

Project ID	**25-26J-011**
Project Title	**RE-MESH**

Student ID	**Name**
IT22571298	P K A Silva
IT22563446	Jayanath D.T.
IT22196088	Q Joyal
IT22617446	D H Lakshitha

Supervisor	Uditha Dharmakeerthi
Co-Supervisor	Amila Senarathne

11/01/2026

DECLARATION OF THE CANDIDATES

We declare that this is our own work, and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of our knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Silva P.K.A.	IT22571298
Jayanath D.T.	IT22563446
Q. Joyal	IT22196088
Lakshitha D.H.	IT22617446

TABLE OF CONTENTS

Declaration of the Candidates i

Abstract **Error! Bookmark not defined.**

Table of Contents iii

List of Abbreviations iv

1. Introduction 1

1.1 Purpose 1

1.2 Scope 1

2. Overall descriptions 3

2.1 Product perspective 3

2.1.1 System interfaces 4

2.1.2 User interfaces 4

2.1.3 Hardware interfaces 5

2.1.4 Software interfaces 5

2.1.5 Communication interfaces 5

2.1.6 Memory constraints 5

2.1.7 Operations 5

2.1.8 Site adaptation requirements 6

2.2 Product Functions 7

2.3 User characteristics 8

2.4 Constraints 8

2.5 Assumptions and dependencies 9

2.6 Apportioning of requirements 9

3. sPEcific requirements 10

3.1 External interface requirements 10

3.1.1 User interfaces 10

3.1.2 Hardware interfaces 12

3.1.3 Software interfaces 13
3.1.4 Communication interfaces 14
3.2 Classes/Objects 14
3.3 Performance requirements 25
3.4 Design constraints 27
3.5 Software system attributes 28
3.5.1 Reliability 28
3.5.2 Availability 29
3.5.3 Security 30
3.5.4 Maintainability 30
3.6 Other Requirements 31
4. Supporting information 32
4.1 Reference List 32

LIST OF ABBREVIATIONS

Abbreviations	Description
API	Application Programming Interface
GPIO	General Purpose Input Output
GPS	Global Positioning System
IoT	Internet of Things
ISM	Industrial, Scientific, and Medical
LoRa	Long Range Radio

1. INTRODUCTION

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to clearly define the functional and non-functional requirements of the RE-MESH system. This document serves as a reference for supervisors, evaluators, developers, and stakeholders to understand the expected behavior, constraints, and performance of the system during the progress presentation and subsequent development phases.

1.2 Scope

This document covers the software requirements for all components of the Re-Mesh system, including:

- Sensor device firmware for water level and environmental monitoring
- ESP32-based gateway firmware for data aggregation and local communication
- LoRa communication protocols for sensor-to-gateway data transmission
- Local Wi-Fi network interface for community access
- End-user web dashboard for real-time flood information access
- Central web dashboard for monitoring, analytics, and emergency management
- Machine learning module for flood risk prediction
- Weather API integration for enhanced prediction accuracy
- GPS-based location tracking for device deployment
- Database management systems for data storage and retrieval
- Alert and notification systems for emergency warnings

Hardware specifications for ESP32 nodes, LoRa modules, and GPS sensors are referenced but detailed in accompanying design document (DD).

1.3 Definitions, Acronyms, and Abbreviations

Term	Description
API	Application Programming Interface
ML	Machine Learning
UI	User Interface
GPS	Global Positioning System
IoT	Internet of Things

ISM	Industrial, Scientific, and Medical
LoRa	Long Range Radio
DMC	Disaster Management Centre (Sri Lanka)

1.4 Overview

RE-MESH is a resilient communication and prediction system designed to maintain connectivity and provide early flood warnings during flood scenarios in Sri Lanka. The system enables people and emergency services to communicate through ESP32-based mesh nodes while simultaneously predicting flood risks using real-time and historical environmental data.

Main Goals:

- Provide early flood warnings to at-risk communities through real-time water level monitoring
- Enable local access to flood information even during internet outages
- Predict flood events using machine learning based on hydrological and environmental data
- Facilitate emergency communication and coordination through local Wi-Fi networks
- Monitor system health and enable rapid device replacement through centralized dashboard

Primary Tasks:

- Continuous water level and environmental data collection from sensors
- LoRa-based data transmission from sensors to gateway devices
- Local Wi-Fi network creation for community access to real-time information
- Flood risk prediction using historical and real-time environmental data
- Emergency alert broadcasting to affected areas
- Centralized monitoring and management of deployed devices

Target Users:

- Communities living near rivers, dams, and flood-prone areas
- Emergency service personnel (rescue teams, medical services, disaster management)
- Local authorities (Irrigation Department, Disaster Management Centre, local councils)

- System administrators monitoring device health and network status
- Decision-makers using flood prediction data for early warning and evacuation planning

2. OVERALL DESCRIPTIONS

The ReMesh system is a comprehensive flood monitoring and emergency communication platform that combines IoT sensor networks, mesh communication, real-time data visualization, and predictive analytics to address critical challenges during flood disasters in Sri Lanka.

The system operates through a network of sensor devices deployed near water bodies that continuously monitor water levels, temperature, humidity, rainfall, and other environmental parameters. These sensors communicate via LoRa protocol with ESP32-based gateway devices, creating a resilient mesh network that functions even when traditional infrastructure fails.

Gateway devices serve dual purposes: they aggregate sensor data and create local Wi-Fi networks that allow nearby residents and emergency responders to access real-time flood information through a simple web interface without requiring internet connectivity. When internet is available, gateways synchronize data to a centralized cloud platform where machine learning models analyze ten years of hydrological data to predict flood risks and generate early warnings.

The integrated machine learning module processes temperature, humidity, water flow, rainfall, turbidity, pH levels, and weather data to forecast flood events 24-72 hours in advance. During emergencies, the system can broadcast critical evacuation messages locally through gateway Wi-Fi networks, ensuring communication continues when cellular networks and internet connections are disrupted.

2.1 Product perspective

ReMesh differentiates itself from existing flood monitoring and emergency communication solutions through its unique combination of decentralized architecture, offline capability, and predictive intelligence specifically designed for Sri Lankan flood scenarios.

Traditional Flood Monitoring Systems: Most existing systems rely on centralized data collection with internet connectivity requirements. These systems often fail during floods when infrastructure is damaged. ReMesh's decentralized approach with local data access ensures continued operation even with complete infrastructure failure.

Commercial IoT Sensor Networks: While commercial solutions provide remote monitoring, they typically require constant cloud connectivity and lack offline functionality. ReMesh prioritizes local-first architecture where data remains accessible to affected communities regardless of internet availability.

Disaster Communication Platforms: Existing platforms focus on post-disaster coordination but don't provide the monitoring infrastructure or predictive capabilities. ReMesh integrates monitoring, prediction, and communication in a single unified system.

Early Warning Systems: Current early warning systems in Sri Lanka depend on manual reporting and centralized decision-making, leading to delays. ReMesh provides automated, data-driven predictions with direct community access to information.

2.1.1 System interfaces

- Sensor device firmware: Embedded C on microcontroller RTOS
- End-user dashboard: Compatible with Windows 10/11, macOS 10.14+, Linux (Ubuntu 20.04+), Android 8+, iOS 12+

2.1.2 User interfaces

Local End-User Dashboard (Offline-Capable):

- Responsive web interface accessible via gateway Wi-Fi connection
- Mobile-first design optimized for smartphones and tablet
- No login required for emergency access

Central Monitoring Dashboard:

- Real-time device status grid with GPS locations
- Interactive map showing sensor and gateway deployment
- Device health metrics (battery, signal strength, data transmission status)
- Device replacement workflow management
- Alert configuration and broadcast controls
- Report generation tools

2.1.3 Hardware interfaces

Sensor Device Components:

- LoRa Transceiver: SPI interface for data transmission
- Power Management: Solar panel with battery backup

Gateway Device (ESP32)

2.1.4 Software interfaces

Machine Learning Framework:

- Python 3.8+ environment
- Pandas (v1.3+) for data manipulation
- NumPy (v1.21+) for numerical operations
- Matplotlib for visualization

Database Management

Weather API Integration

Web Framework

2.1.5 Communication interfaces

LoRa Sensor-to-Gateway Protocol

Wi-Fi Local Network (Gateway Access Point)

GPS Communication

2.1.6 Memory constraints

Sensor Device

Gateway Device

Central Server

2.1.7 Operations

Community User Operations:

- Connect to gateway Wi-Fi network
- Access local dashboard automatically (captive portal)
- View current water level and flood risk

- Check environmental conditions
- Access emergency contact information
- No account or authentication required

Emergency Personnel Operations:

- Access central dashboard with authentication
- Monitor water levels across all locations
- Review flood predictions and risk assessments
- Issue emergency alerts and evacuation orders
- Broadcast messages to specific areas
- Track device status and coverage
- Coordinate response activities
- Generate incident reports

Administrator Operations:

- Monitor all sensors and gateways on central dashboard
- View GPS locations of deployed devices
- Identify failed or low-battery devices
- Initiate device replacement procedures
- Configure alert thresholds and rules
- Review prediction accuracy and system performance

2.1.8 Site adaptation requirements

Geographic Deployment:

- System supports GPS coordinates within Sri Lankan boundaries
- Device placement optimized for Sri Lankan river systems and dams
- Flood prediction models trained on Sri Lankan hydrological patterns

Regulatory Compliance:

- LoRa frequency selection compliant with TRCSL regulations
- ISM band usage within legal transmission power limits

- Data privacy compliance with Sri Lankan data protection laws
- Integration compatible with Disaster Management Centre protocols
- Coordination with Irrigation Department monitoring systems

Environmental Adaptation:

- Waterproof enclosures (IP67 minimum) for monsoon resilience
- Operating temperature: 15°C to 45°C
- Humidity tolerance: 10% to 98% (tropical climate)
- UV-resistant materials for outdoor deployment
- Corrosion-resistant for high-humidity environments
- Solar panel efficiency optimized for tropical sunlight

Language and Cultural Adaptation:

- Emergency terminology appropriate for Sri Lankan context
- Color coding aligned with local emergency alert standards
- Evacuation route information integrated with local geography

2.2 Product Functions

Real-Time Environmental Monitoring:

- Continuous water level measurement at sensor locations
- Temperature and humidity monitoring
- Rainfall measurement and accumulation tracking
- pH and turbidity monitoring (where applicable)
- Data collection frequency: Every 5-10 minutes
- Local data buffering when gateway unavailable
- Automatic data quality validation

LoRa-Based Data Transmission

- Long-range wireless transmission from sensors to gateways (2-10 km)
- Low-power operation for extended battery life
- Automatic retry mechanism for failed transmissions
- Data encryption for security

- Multi-sensor support per gateway
- Signal strength monitoring and reporting

Local Wi-Fi Information Access:

- Gateway creates local Wi-Fi network for community access
- Offline-capable web dashboard hosted on gateway
- Real-time water level display without internet requirement
- Current conditions and trends accessible locally
- Emergency alert display on local dashboard
- Automatic connection via captive portal
- No authentication required for emergency access

Flood Risk Prediction Using Machine Learning

- Collect and process 10 years of historical hydrological data
- Real-time data integration (water level, rainfall, weather, flow)
- XGBoost and Logistic Regression model training with hyperparameter tuning
- Predict flood probability for 24, 48, and 72-hour timeframes
- Model confidence scoring
- Continuous model improvement with new data
- Feature importance analysis for interpretability

2.3 User characteristics

General public: basic technical knowledge

Emergency service personnel: trained users

System administrators: technical expertise required

2.4 Constraints

Limited power availability during floods

Hardware resource limitations

Environmental conditions affecting sensors

2.5 Assumptions and dependencies

Sensors provide accurate data

Weather API availability

Sufficient historical data for ML training

2.6 Apportioning of requirements

Phase 1: Mesh communication and dashboards

Phase 2: Flood prediction model integration

Phase 3: Optimization and validation

3. SPECIFIC REQUIREMENTS

3.1 External interface requirements

3.1.1 User interfaces

Local End-User Dashboard (Offline-Capable):

Provide community access to real-time flood information via gateway Wi-Fi without internet requirement

Layout and Components:

Main Display Area:

- Large water level gauge with color-coded zones
- Green zone: Safe (< 50% capacity)
- Yellow zone: Caution (50-75% capacity)
- Orange zone: Warning (75-90% capacity)
- Red zone: Danger (> 90% capacity)
- Current water level (meters) and percentage of capacity
- Trend indicator (rising/falling/stable) with arrow icons
- Time since last update

Alert Banner:

- Prominent alert notification area (collapsible)
- Color-coded by severity (blue/yellow/red)
- Alert message in plain language
- Timestamp of alert
- Recommended actions (evacuation instructions)

Emergency Information:

- Emergency contact numbers (fire, ambulance, disaster management)
- Nearest safe location information

User Interaction:

- User connects to "ReMesh-Gateway-[Location]" Wi-Fi
- Browser automatically opens dashboard (captive portal)

- User sees current water level and conditions immediately
- Alerts displayed prominently if active
- No login required; immediate access

Central Monitoring Dashboard:

Comprehensive monitoring and management interface for administrators and emergency personnel

Layout and Components:

System Overview Panel:

- Total devices deployed
- Active devices count
- Failed/offline devices count
- Current overall flood risk level (Low/Moderate/High/Critical)
- Active alerts count

Geographic Map View (Primary):

- Interactive map
- Gateway markers color-coded by status
- Sensor device markers clustered around gateways
- Water level heatmap overlay (optional)
- Coverage area circles
- Click marker for device details popup
- Zoom and pan controls
- Layer toggles (gateways, sensors, alerts, coverage)

Device List Table:

- Sortable and filterable columns
- Columns: Device ID, Location, Type, Status, Battery %, Signal, Water Level, Last Seen, Actions
- Search and filter controls
- Pagination for large deployments
- Export to CSV button
- Bulk action selection

Alert Management Section:

- Active alerts list with severity and location
- "Create Alert" button
- Alert templates dropdown
- Geographic targeting selector
- Broadcast history
- Alert acknowledgment status

Device Detail View (Modal/Side Panel):

- Device ID and name
- GPS coordinates and map
- Installation date
- Current status details
- Battery voltage and charge status
- Solar panel status
- Signal strength (RSSI)
- Data transmission log
- Recent sensor readings (table)
- Configuration settings
- Maintenance history
- "Replace Device" workflow button
- "Configure Device" button
- Diagnostic logs viewer

3.1.2 Hardware interfaces

ESP32 Microcontroller Interface Specifications:

LoRa Module:

- Interface Type: SPI (Serial Peripheral Interface)
- Pins Used:
    - MOSI (GPIO 23): Data from ESP32 to LoRa
    - MISO (GPIO 19): Data from LoRa to ESP32
    - SCK (GPIO 18): Clock signal
    - CS (GPIO 5): Chip select

- RST (GPIO 14): Reset
- DIO0 (GPIO 26): Interrupt for received packets
- SPI Clock Speed: 10 MHz

GPS Module

Environmental Sensors:

- Water Level Sensor: Ultrasonic or pressure-based, analog/digital interface
- Temperature/Humidity Sensor: Single-wire digital interface
- Rainfall Sensor: Tipping bucket, digital pulse counter
- pH Sensor (optional): Analog interface with calibration
- Turbidity Sensor (optional): Analog interface

Power Management:

3.1.3 Software interfaces

Machine Learning Framework:

- Python Environment:
  Package Manager: conda
- Data Processing Libraries:
  Pandas >= 1.3.0: Data manipulation and analysis
  NumPy >= 1.21.0: Numerical computations
  Matplotlib >= 3.4.0: Data visualization
- Model Serialization:
  Library: pickle
  Format: .pkl file
  Storage: Local filesystem with version control

Database Management

Weather API Integration

- Service Provider: OpenWeatherMap API
- Request Parameters:
  - lat: Latitude of node location
  - lon: Longitude of node location

- appid: API authentication key
- units: metric (for Celsius and metric measurements)

- Response Format: JSON

3.1.4 Communication interfaces

LoRa Sensor-to-Gateway Protocol

Wi-Fi Local Network (Gateway Access Point)

GPS Communication

3.2 Classes/Objects

Node Management Classes:

Class: Node [Essential]

- Attributes:

  - nodeId: String (unique identifier)

  - location: GPSCoordinate (latitude, longitude, altitude)

  - status: Enum (Active, Inactive, Warning, Failed)

  - batteryLevel: Float (0-100%)

  - signalStrength: Integer (RSSI in dBm)

  - firmwareVersion: String

  - lastSeen: Timestamp

  - connectedUsers: Integer

  - messageCount: Integer

  - uptime: Integer (seconds)

- Methods:

  - updateStatus(): Update node status based on health checks

  - getBatteryLevel(): Query battery voltage and calculate percentage

  - sendHeartbeat(): Broadcast status to central server

- handleIncomingMessage(message): Process received LoRa messages
- forwardMessage(message): Route message to next hop

Class: GPSCoordinate [Essential]

- Attributes:
  - latitude: Float
  - longitude: Float
  - altitude: Float
  - accuracy: Float (meters)
  - timestamp: Timestamp
- Methods:
  - distanceTo(otherCoordinate): Calculate distance between coordinates
  - isValid(): Check if coordinates are within valid range
  - toDecimalDegrees(): Convert from NMEA format
  - toString(): Format for display

Class: MeshNetwork [Essential]

- Attributes:
  - nodes: List<Node>
  - routingTable: Map<nodeId, Route>
  - networkId: String
- Methods:
  - discoverNodes(): Perform neighbor discovery
  - buildRoutingTable(): Calculate optimal routes

- findRoute(destinationId): Get route to destination
- handleNodeFailure(nodeId): Reconfigure network on node failure
- optimizeTopology(): Adjust network for better coverage

Messaging Classes:

Class: Message [Essential]

- Attributes:
  - messageId: String (UUID)
  - senderId: String
  - recipientId: String (or "BROADCAST")
  - content: String (max 200 characters)
  - timestamp: Timestamp
  - priority: Enum (Emergency, Normal, Low)
  - deliveryStatus: Enum (Queued, InTransit, Delivered, Failed)
  - hops: Integer (number of relays)
  - encryptedPayload: Bytes
- Methods:
  - encrypt(): Encrypt message content
  - decrypt(): Decrypt received message
  - validate(): Check message format and size
  - toLoRaPacket(): Convert to LoRa transmission format
  - fromLoRaPacket(packet): Parse received packet

Class: MessageQueue [Essential]

- Attributes:
  - queue: PriorityQueue<Message>
  - maxSize: Integer (100)
- Methods:
  - enqueue(message): Add message to queue
  - dequeue(): Get next message by priority
  - peek(): View next message without removing
  - clear(): Remove all messages
  - getQueueStatus(): Return queue size and oldest message age

Class: MessageRouter [Essential]

- Attributes:
  - routingTable: Map<destinationId, nextHop>
  - messageCache: Set<messageId> (prevent duplicates)
- Methods:
  - routeMessage(message): Determine next hop for message
  - cacheMessage(messageId): Store message ID to prevent loops
  - isMessageCached(messageId): Check for duplicate
  - updateRoute(destination, nextHop): Update routing entry

User Interface Classes

Class: EndUserDashboard [Essential]

- Attributes:

  - userId: String

  - connectedNode: Node

  - messageList: List<Message>

  - networkStatus: ConnectionStatus

- Methods:

  - renderMessageList(): Display user messages

  - sendMessage(content, recipient): Submit new message

  - refreshMessages(): Fetch new messages from node

  - displayNetworkStatus(): Show connection quality

  - handleEmergencyButton(serviceType): Send emergency alert

Class: CentralMonitoringDashboard [Essential]

- Attributes:

  - nodeList: List<Node>

  - mapView: MapComponent

  - alertList: List<Alert>

  - currentUser: AdminUser

- Methods:

  - renderNodeMap(): Display geographic node distribution

  - updateNodeStatus(nodeId, status): Refresh node information

  - generateReport(dateRange): Create performance report

  - triggerAlert(alertType, message): Create system alert

  - filterNodes(criteria): Apply filters to node list

Class: EmergencyServiceDashboard [Desirable]

- Attributes:
  - serviceType: Enum (Medical, Fire, Police, Rescue)
  - priorityQueue: MessageQueue
  - activeIncidents: List<Incident>
  - resources: List<Resource>
- Methods:
  - broadcastMessage(content, zone): Send to all users in area
  - viewPriorityMessages(): Display urgent messages first
  - assignResource(incidentId, resourceId): Allocate personnel/equipment
  - updateIncidentStatus(incidentId, status): Track incident progress

Flood Prediction Classes

Class: FloodPredictor [Essential]

- Attributes:
  - model: XGBoostClassifier or LogisticRegression
  - featureScaler: StandardScaler
  - modelVersion: String
  - lastTrainingDate: Timestamp
  - accuracy: Float
- Methods:
  - predict(sensorData): Generate flood risk prediction
  - predictBatch(dataList): Process multiple predictions
  - evaluateModel(testData): Calculate performance metrics

- getFeatureImportance(): Return important features
- saveModel(filepath): Serialize model to disk
- loadModel(filepath): Load trained model

Class: HydrologicalData [Essential]

- Attributes:
  - timestamp: Timestamp
  - temperature: Float (°C)
  - humidity: Float (%)
  - waterFlow: Float (m^3/s)
  - rainfall: Float (mm)
  - turbidity: Float (NTU)
  - pH: Float
  - weatherCondition: String
  - floodRisk: Boolean (label for training)
- Methods:
  - toFeatureVector(): Convert to ML input format
  - validate(): Check data quality and ranges
  - normalize(): Apply feature scaling
  - fillMissingValues(): Impute missing data

Class: ModelTrainer [Essential]

- Attributes:
  - trainingData: List<HydrologicalData>

  - validationData: List<HydrologicalData>

  - testData: List<HydrologicalData>

  - hyperparameters: Dict

- Methods:

  - loadData(filepath): Import historical data

  - preprocessData(): Clean and prepare data

  - splitData(trainRatio, valRatio): Divide dataset

  - tuneHyperparameters(): Perform grid/random search

  - trainXGBoost(): Train XGBoost model

  - trainLogisticRegression(): Train logistic regression model

  - compareModels(): Evaluate and select best model

  - crossValidate(folds): Perform k-fold cross-validation

Class: WeatherDataCollector [Essential]

- Attributes:

  - apiKey: String

  - apiEndpoint: String

  - updateInterval: Integer (seconds)

  - lastUpdate: Timestamp

- Methods:

  - fetchWeatherData(location): Retrieve current weather

  - parseAPIResponse(response): Extract relevant fields

  - storeData(weatherData): Save to database

  - scheduleUpdates(): Set up periodic data collection

  - handleAPIError(error): Retry logic and fallback

Data Management Classes

Class: DatabaseManager [Essential]**

- Attributes:

  - postgresConnection: Connection

  - mongoConnection: Connection

  - redisConnection: Connection

- Methods:

  - saveMessage(message): Store message in PostgreSQL

  - getMessages(userId, limit): Retrieve user messages

  - saveNodeStatus(nodeId, status): Log node status

  - saveSensorReading(sensorData): Store in MongoDB

  - cacheData(key, value, ttl): Store in Redis

  - getCachedData(key): Retrieve from cache

Class: DataLogger [Desirable]

- Attributes:

  - logLevel: Enum (Debug, Info, Warning, Error)

  - logFile: String

  - maxLogSize: Integer

- Methods:

  - logInfo(message): Log informational message

  - logError(error): Log error with stack trace

- logWarning(warning): Log warning
- rotateLog(): Archive old logs when size exceeded
- queryLogs(criteria): Search log entries

Alert Management Classes

Class: Alert [Desirable]

- Attributes:
  - alertId: String
  - type: Enum (NodeFailure, LowBattery, HighFloodRisk, NetworkCongestion)
  - severity: Enum (Info, Warning, Critical)
  - message: String
  - timestamp: Timestamp
  - acknowledged: Boolean
  - affectedNodes: List<String>
- Methods:
  - trigger(): Send alert to administrators
  - acknowledge(userId): Mark alert as seen
  - resolve(): Close alert
  - escalate(): Increase severity level

Class: AlertManager [Desirable]

- Attributes:
  - activeAlerts: List<Alert>
  - alertRules: List<AlertRule>

- notificationChannels: List<NotificationChannel>

- Methods:

  - evaluateRules(): Check conditions for triggering alerts

  - createAlert(type, message, severity): Generate new alert

  - sendNotification(alert, channels): Distribute alert

  - getActiveAlerts(): Retrieve unresolved alerts

  - configureRule(rule): Add/update alert rule

3.3 Performance requirements

Message Delivery Performance

Latency Requirements:

- Node-to-node direct communication: < 1 second per hop
- End-to-end message delivery (max 5 hops): < 10 seconds under normal conditions
- Message queue processing: < 100 ms per message
- Dashboard message refresh: < 2 seconds

Throughput Requirements:

- Per node message capacity: Minimum 10 messages per minute
- Network aggregate: 1000+ messages per hour with 50 active nodes
- Central server API: 500 requests per second

Reliability:

- Message delivery success rate: > 95% under normal conditions
- Message delivery success rate: > 80% during node failures
- Duplicate message rate: < 1%
- Message ordering: Best effort (not guaranteed for multi-hop)

Flood Prediction Performance

Prediction Speed:

- Real-time prediction (single input): < 500 ms
- Batch prediction (100 inputs): < 5 seconds
- Model loading time: < 2 seconds on server startup

Training Performance:

- Initial model training (10 years data, ~87,600 records): < 30 minutes
- Hyperparameter tuning (grid search): < 4 hours
- Incremental retraining (monthly): < 10 minutes

Accuracy Requirements:

- Minimum classification accuracy: > 85%
- Target accuracy: > 90%
- False positive rate: < 15% (avoid unnecessary panic)
- False negative rate: < 5% (critical to catch actual floods)
- ROC AUC score: > 0.90

Data Processing:

- Weather API response parsing: < 100 ms
- Sensor data preprocessing: < 200 ms for 24 hours of data
- Feature engineering: < 1 second

Dashboard Performance

Page Load Time:

- Initial page load: < 3 seconds on 3G connection
- Dashboard interaction response: < 300 ms
- Map rendering: < 2 seconds for 100+ nodes
- Chart rendering: < 1 second

Resource Usage:

- Browser memory: < 150 MB for end-user dashboard
- Browser memory: < 300 MB for central monitoring dashboard
- CPU usage: < 30% on mid-range devices

- Mobile data usage: < 5 MB per hour of active use

Scalability:

- Support for 100+ nodes on central dashboard

- Support for 1000+ messages in history

- Support for 50+ simultaneous end-users

- Support for 20+ simultaneous administrators

System Uptime and Availability

Node Availability:

- Individual node uptime: > 99% (excluding intentional maintenance)

- Network availability (at least 60% nodes operational): > 99.5%

- Battery life: Minimum 7 days without solar charging

Server Availability:

- Central server uptime: > 99.9% (< 9 hours downtime per year)

- Database availability: > 99.9%

- API availability: > 99.5%

Recovery Time:

- Node reboot time: < 30 seconds

- Network reconfiguration after node failure: < 2 minutes

- Server recovery from crash: < 5 minutes

- Database backup restore: < 1 hour

3.4 Design constraints

Hardware Constraints

ESP32 Limitations:

- Flash memory: 4 MB total (firmware must fit in 2 MB, leaving 2 MB for data)
- RAM: 520 KB (application must use < 400 KB to leave margin)
- Processing power: Dual-core 240 MHz (limited for complex encryption or ML)
- Power consumption: Must optimize for battery operation (sleep modes required)
- GPIO pins: Limited number requires careful allocation

Regulatory Constraints

Telecommunications:

- LoRa frequency must comply with Sri Lankan TRCSL regulations
- Transmission power limits as per ISM band rules
- Licensing requirements for deployment (if applicable)

Data Privacy:

- Must comply with Sri Lankan data protection regulations
- User message privacy and encryption requirements
- Data retention and deletion policies
- User consent for data collection

Network Architecture Constraints

Mesh Topology:

- Decentralized design required (no single point of failure)
- Self-healing capability essential
- Maximum 5 hops to prevent excessive latency

Internet Dependency:

- End-user messaging must work without internet
- Central monitoring requires internet (can operate with intermittent connectivity)
- Weather API requires internet (must cache for offline operation)

3.5 Software system attributes

3.5.1 Reliability

Message Delivery Reliability:

- Acknowledgment mechanism for critical messages

- Automatic retry up to 3 times for failed transmissions

- Message persistence in queue until delivered or expired

- Store-and-forward capability when routes unavailable

- Duplicate detection to prevent message replication

- Target: 95% successful delivery rate under normal conditions

System Fault Tolerance:

- Graceful degradation when nodes fail (network continues with reduced capacity)

- No single point of failure in mesh network

- Automatic route recalculation when nodes go offline

- Central server redundancy (primary and backup server recommended)

- Database replication for data safety

- Automatic recovery from transient failures

Data Integrity:

- CRC checks on all LoRa transmissions

- Database transactions with ACID properties

- Data validation at input and storage layers

- Checksums for firmware updates

- Backup verification before restoration

3.5.2 Availability

System Availability Targets:

- Node availability: 99% uptime (87 hours downtime per year)

- Network availability (minimum viable mesh): 99.5% uptime

- Central server: 99.9% uptime (8.76 hours downtime per year)

- Flood prediction service: 99% availability

- End-user dashboard access: 99% when connected to node

Availability Strategies:

Hardware Redundancy:

- Mesh network inherent redundancy (multiple nodes provide coverage)

- Dual-power system (solar + battery backup)

- Spare nodes pre-deployed in critical areas

Software Redundancy:

- Load balancing for central server (if using multiple servers)

- Database replication (master-slave or multi-master)

3.5.3 Security

Authentication and Authorization:

User Authentication:

- End-users: No authentication required for basic messaging (emergency access)

- Emergency services: Username/password authentication

- Administrators: Multi-factor authentication (MFA) recommended

- Session management with secure tokens

- Session timeout: 30 minutes inactivity for admin, 4 hours for emergency services

3.5.4 Maintainability

Code Maintainability:

Code Quality:

- Consistent coding style (enforced by linters)

- Meaningful variable and function names

- Maximum function length: 50 lines

- Maximum file length: 500 lines

- Code complexity limits (cyclomatic complexity < 10)

Documentation:

- Inline code comments for complex logic

- Function/class documentation with parameters and return values

- API documentation (Swagger/OpenAPI)

- Architecture diagrams (system, network, data flow)

- Database schema documentation

- Deployment procedures documented

- Troubleshooting guides

System Monitoring:

Logging:

- Structured logging (JSON format)

- Log levels: DEBUG, INFO, WARNING, ERROR, CRITICAL

- Centralized log aggregation

- Log rotation to prevent disk space issues

- Searchable logs with indexing

3.6 Other Requirements

The system shall support future extensions including machine learning-based flood prediction, emergency text or voice message dissemination, and enhanced energy optimization techniques.

4. SUPPORTING INFORMATION

4.1 Reference List

[1] S. G. R. Perera, J. Wijesinghe, and R. L. Liyanage, "Flood hazard mapping and risk assessment in Sri Lanka," *Natural Hazards*, vol. 103, no. 2, pp. 1–18, 2020.

[2] Disaster Management Centre (DMC) Sri Lanka, *National Disaster Management Plan*, Ministry of Defence, Sri Lanka, 2022.

[3] Sri Lanka Telecommunications Regulatory Commission (TRCSL), *Radio Frequency Spectrum Allocation and ISM Band Regulations*, TRCSL, Colombo, Sri Lanka, 2023.

[4]"Irrigation department in Sri Lanka," *Gov.lk*. [Online]. Available: https://www.irrigation.gov.lk/web/index.php?lang=en. [Accessed: 02-Jan-2026].